Runtime Evidence for Agent-Ready Command-Line Interfaces

research@modiqo.ai

June 13, 2026

Abstract

Command-line interfaces are becoming a primary substrate for software agents, build harnesses, and automation systems. They are already installed in developer environments, carry existing authentication state, expose operational authority, and provide a stable bridge between local workflows and remote systems. Yet most CLIs are not measured as machine-operable interfaces. Their discoverability, grammar, output contracts, precondition behavior, and safety properties are usually inferred informally from help text, documentation, examples, and repeated failure.

CLIARE treats a CLI as a black-box runtime system and measures it through the executable as shipped. The system records runtime evidence, infers a probabilistic command-shape catalog, classifies precondition-blocked paths, observes filesystem side effects under bounded probes, and emits scorecards and artifact maps suitable for CI, drift detection, agent navigation, and improvement tracking. This paper presents the motivation, evidence model, inference architecture, score semantics, calibration boundary, and early evaluation strategy for CLIARE. The central claim is simple: CLI-agent-readiness should be a measured property of the released executable, not an assertion in documentation.

1 Introduction

Modern automation is increasingly mediated by command-line tools. Source control, package management, deployment, database operations, cloud control planes, test runners, project scripts, and internal platform workflows are commonly exposed through CLIs. Agents inherit that ecosystem. They use CLIs because CLIs are available where work happens: in repositories, shells, CI workers, ephemeral containers, developer laptops, and release pipelines.

This creates an interface quality problem. Human users can compensate for ambiguous help output, missing examples, inconsistent exit codes, unstructured output, interactive prompts, and undocumented preconditions. Agents and harnesses have a narrower operating envelope. They need to discover commands, construct invocations, parse responses, recover from errors, avoid destructive actions, and detect drift without relying on stale assumptions.

The current ecosystem has strong conventions for API schemas, test coverage, static analysis, supply-chain provenance, and benchmark reporting. CLIs have weaker measurement infrastructure, despite long-standing command-line conventions such as POSIX utility syntax guidance [**?**]. A CLI may be excellent for an experienced human and poor for an agent. Another may be small but highly regular, with stable JSON output and clear diagnostics. Without a runtime scorecard, both cases are hard to compare and hard to improve.

CLIARE closes that gap without requiring source-code access, assuming a parser framework, or asking maintainers to upload binaries to a hosted runner. It executes bounded probes against the installed binary, records what happened, and produces artifacts that can be consumed by maintainers, CI systems, agent harnesses, and calibration workflows.

2 Problem Setting

Let a CLI be a runtime program B that maps an argument vector, environment, working directory, filesystem state, terminal state, and external preconditions into process observations:

$$B(\mathbf{a}, \eta, d, F, \tau, N, A) \to o.$$

Here $\mathbf{a}$ is the argument vector, η is the environment, d is the working directory, F is the filesystem state, τ is the terminal state, N is the network state, A is the authentication and precondition state, and o is the resulting observation.

An observation can include exit status, stdout, stderr, duration, created files, modified files, deleted files, spawned process behavior, and classified diagnostics. The true command surface is latent. CLIARE cannot assume that help text is complete, that completion scripts are current, that documented flags are accepted, or that source-code-level command declarations match the installed executable.

The target measurement problem is to estimate how well an agent can operate B under realistic constraints. The measurement must account for discovery, grammar, execution, output, recovery, safety, preconditions, and drift.

Runtime context is part of the measurement. A clean unauthenticated process, an authenticated process, and a process launched inside a project workspace may expose different reachable command surfaces even when the binary fingerprint is identical. CLIARE models this as a context variable C and scopes evidence accordingly:

$$S(B, C) = 100 \cdot \mathbb{E}[U(G, T, C) \mid E_C].$$

Here E_C is the evidence gathered under context C. Context-suite artifacts compare those per-context measurements without flattening persona reports, command indexes, or evidence logs into one directory.

The difficulty is not only parsing. Help output is only one evidence source, and in many CLIs it is incomplete, inconsistent, paginated, localized, generated from stale templates, gated by authentication, or emitted in manpage-like layouts. A serious extractor must treat help as evidence, not truth.

3 Design Requirements

CLIARE is built around six design requirements.

1. Measurement must be black-box. Maintainers should be able to measure the released executable that users and agents actually run.

2. Inference must be framework-agnostic. CLIs may use clap, cobra, click, argparse, oclif, shell scripts, hand-written parsers, plugin dispatchers, or legacy conventions.

3. Every artifact must carry evidence. Shape fields must retain confidence, evidence references, runtime state, and model version.

4. The score must be decomposable. Discovery, grammar, execution, output, safety, and recovery should be visible independently.

5. The system must run in the maintainer's CI environment. Arbitrary CLI execution is a security boundary.

6. Public ranking must be calibrated. A leaderboard score requires truth sets, proper scoring metrics, repeated-run stability, confidence intervals, and governance for score-model changes.

4 System Overview

The CLIARE pipeline is:

```
target binary
  -> fingerprint
  -> bounded probe scheduler
  -> sandboxed process execution
  -> append-only evidence log
  -> layout and diagnostic observations
  -> probabilistic claim store
  -> command-shape catalog
  -> scorecard, reports, and artifact map
```

The current reference implementation emits an evidence log, command-shape catalog, command index, issue ledger, persona packets, scorecard, report, CI summary, SARIF findings, JUnit XML, runtime context metadata, context-suite comparisons, and an optional artifact map generated by `cliare describe <folder>`. The artifact map is not a scoring input. It is a directory-level manifest that records artifact kind, file roles, schema versions, missing required artifacts, current job state, and recommended navigation order for agents and humans. CI use is intentionally direct:

```
cliare measure ./target/debug/mycli --profile standard
cliare guard ./target/debug/mycli --baseline .cliare/baseline.scorecard.json
```

The same mechanism measures CLIARE itself. That self-measurement is not a special framework path; it is a regression check that the generic processor can operate on a clean modern CLI surface.

5 Evidence Model

CLIARE separates observations from claims. Observations are facts about probes. Claims are inferred statements about the CLI.

```
probe p_0009 ran: cliare metadata --format json --help
exit status: 0
stdout parsed as JSON
duration: 7 ms
filesystem changes: none
```

A claim derived from that observation might state that the `metadata` command advertises and honors a JSON output mode through `--format json`. The claim carries confidence, model version, and evidence references.

This distinction allows CLIARE to rescore old evidence under a new model, inspect contradictions, explain claims, and avoid treating one parser pass as ground truth. Evidence is gathered from safe bootstrap probes first, then later probes are scheduled from candidate claims. The scheduler ranks probes by expected information value subject to risk, cost, depth, and probe-budget constraints.

6 Inference Model

CLIARE uses layered inference. Raw text becomes layout observations. Layout observations produce candidate claims. Runtime probes update belief. Shape emission happens only after confidence is computed.

The generic layout processor considers indentation, blank-line grouping, repeated row alignment, compact invocation cells, token morphology, metavariables, dash-prefixed flags, continuation likelihood, and manpage-like formatting. It does not decide that a row is a command solely because a nearby heading says `Commands`. Section titles are weak features. Runtime confirmation carries more weight.

For binary claims, the current reference model uses log-odds updates:

$$\operatorname{logit} P(z \mid E) = \operatorname{logit} P(z) + \sum_i w_i.$$

$$P(z \mid E) = \sigma(\operatorname{logit} P(z \mid E)).$$

This is equivalent to multiplying prior odds by evidence likelihood ratios:

$$\frac{P(z \mid E)}{1 - P(z \mid E)} = \frac{P(z)}{1 - P(z)} \prod_i \exp(w_i).$$

The model is deliberately compact. It is explainable, deterministic for a fixed evidence set, and easy to calibrate later against truth sets. Candidate command rows provide weak positive evidence. Runtime help confirmation provides strong positive evidence. A clean unknown-command diagnostic provides negative evidence. A runtime precondition diagnostic is not command absence; it is evidence that the command was recognized but blocked by a condition such as authentication, local workspace context, required operands, fixture data, or runtime dependencies.

This treatment is important for real CLIs. A command whose help is gated by login, profile, current directory, or plugin installation should not be counted as nonexistent. It should be represented as precondition-blocked runtime state with the corresponding required precondition. Diagnostics that include labeled fixes, command examples, hints, or help references improve recovery quality because they make the precondition navigable for agents and maintainers.

7 Scoring Model

The reference implementation currently publishes `cliare-score-v0`. It is intended for local measurement, CI regression checks, and release-to-release improvement tracking. It is not yet the frozen public leaderboard model.

The formal target is posterior expected utility:

$$S(B) = 100 \cdot \mathbb{E}[U(G, T) \mid E].$$

Here G is the latent true command surface, T is a task or workload distribution, E is observed runtime evidence, and U is an agent-readiness utility function.

The v0 implementation approximates that target with deterministic subscores:

$$S_{\text{v0}} = \frac{\sum_{d \in D} \omega_d s_d}{\sum_{d \in D} \omega_d}.$$

Here D is the set of measured score dimensions, s_d is the subscore for dimension d, and ω_d is its model-versioned weight.

Dimension	Purpose
Discovery	Can an agent find and recognize the command surface?
Grammar	Can an agent construct valid invocations?
Execution	Do probes terminate without hangs or spawn failures?
Recovery	Do invalid invocations fail cleanly?
Output	Are machine-readable modes advertised and parseable?
Safety	Do safe probes avoid persistent side effects?

The score is designed to move when maintainers make concrete improvements: clearer help, reachable subcommand help, explicit flag arity, parseable JSON output, stable exit codes, noninteractive behavior, precise auth diagnostics, and read-only metadata paths.

Whole-surface scores can drop when a CLI adds many poorly documented commands. That is expected. The surface grew, and the new surface is not yet as agent-ready as the previous one. For this reason CLIARE distinguishes whole-surface scoring from known-surface regression analysis.

8 Execution, Safety, and Reproducibility

Black-box measurement is only useful if execution is bounded and evidence is reproducible. CLIARE probes run with timeout limits, output limits, null stdin, isolated working directories, isolated HOME, isolated XDG config/cache/data directories, and a sanitized environment. Filesystem snapshots around each probe classify created, modified, and deleted files.

Safety scoring in v0 focuses on persistent side effects during discovery probes. This is intentionally conservative and does not claim to solve destructive-action analysis completely. The initial goal is to detect whether help, version, metadata, and diagnostic paths are read-only. A CLI that writes credentials, mutates project files, or creates uncontrolled cache files during discovery is less safe for agents and CI.

CLIARE also fingerprints the target binary and measurement profile. Cache reuse is valid only when the target fingerprint, runtime context, traversal profile, sandbox profile, resolved probe budget, expected-value threshold, concurrency limit, CLIARE version, measurement engine, and artifact set match. Otherwise probes are rerun. This prevents a clean run from satisfying an authenticated or workspace-context measurement.

This design aligns with supply-chain provenance principles: scorecards should be traceable to the binary, build, profile, runner, and evidence that produced them [**?**]. CLIARE does not need to execute third-party binaries in a hosted cloud service to publish useful results; it can publish scorecards and provenance from the maintainer's own CI.

9 CI Integration and Publishing

CLIARE's intended deployment point is the release pipeline. Pull requests can run a quick profile, main-branch builds can run the standard profile, and scheduled jobs can run deeper traversal. A guard mode compares current results to an accepted baseline and fails when score regressions exceed policy.

A public scorecard should include target name and version, binary fingerprint, CLIARE version, score model version, traversal profile, sandbox profile, probe counts, stop reason, subscores, findings,

output-contract coverage, side-effect evidence, precondition-blocked counts, artifact hashes, and verification level.

Publishing should be scorecard-first, not binary-first. The hosted layer can display trends, badges, leaderboards, and historical drift without requiring arbitrary CLI execution in modiqo infrastructure. Certified public ranking is a later layer that requires calibrated profiles and verification controls.

10 Preliminary Evaluation

The current reference implementation has been exercised against synthetic fixtures and a local real-CLI corpus. The synthetic suite covers command inference, runtime false-positive rejection, precondition-blocked help, cache reuse, guard policies, sandbox isolation, parseable output, malformed output, clean probes, cache writes, credential-like writes, SARIF, JUnit, CI summaries, and serial-versus-concurrent traversal equivalence.

Target	Score	Probes	Disc.	Conf.	Contracts	Parses	Findings	Side effects
cliare	97.4	23	5	5	1	1	0	0

Table 1: Standard-profile CLIARE-on-CLIARE run. Traversal completed under the standard profile.

This result is not presented as a benchmark claim about the ecosystem. It is a sanity check for the reference implementation: the tool can measure itself through the generic path, identify a parseable metadata contract, avoid false output-contract discoveries, and complete traversal under the standard profile.

10.1 Exploratory Real-CLI Corpus

A local corpus run was also performed against CLIARE, rote, git, supabase, gh, cargo, npm, docker, and deno. These measurements are not presented as a public ranking. They are included to show that the measurement pipeline exercises heterogeneous real CLI surfaces and exposes pressure signals before certification calibration.

The run used `cliare-score-v0`, the local benchmark manifest, target concurrency 3, and the maintainer's local environment on 2026-06-13. It measured 9 targets, skipped 0, failed 0, completed 2703 probes, observed a 66.7% traversal completion rate, and exhausted the probe budget on 33.3% of targets. It also observed 19 precondition-blocked probes.

The table is most useful when read by column, not as a leaderboard. Probe-budget exhaustion on rote, gh, and deno indicates large or branching surfaces that need deeper profiles or improved convergence. Side-effect files on rote, npm, and deno show why discovery-time filesystem evidence matters. The gap between discovered machine-readable contracts and parse successes shows where safe output-mode probing and command-specific fixtures need to mature. These are engineering signals, not certified quality judgments.

11 Calibration Boundary

The difference between a useful CI score and an authoritative public score is calibration.

For public ranking, CLIARE must add human-reviewed truth sets for synthetic and real CLIs, proper scoring metrics for binary and categorical claims, expected calibration error for confidence values, repeated-run stability across clean environments, false-safe-rate reporting for safety classification, certified traversal profiles, provenance and verification levels, and score-model governance.

The calibration workflow should evaluate whether confidence means what it says. If CLIARE assigns 0.8 probability to a set of command-existence claims, roughly 80 percent should be true under the labeled truth set. Brier score, log loss, expected calibration error, and false-safe rate should be reported by model version [**?**, **?**, **?**].

For binary claims, a basic Brier score over n labeled claims is:

$$\text{Brier} = \frac{1}{n}\sum_{i=1}^{n}(p_i - y_i)^2,$$

where p_i is the predicted probability and $y_i \in \{0, 1\}$ is the truth label.

Expected calibration error over M confidence bins can be reported as:

$$\text{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left|\text{acc}(B_m) - \text{conf}(B_m)\right|.$$

Here B_m is a bin of claims, $\text{acc}(B_m)$ is the empirical accuracy in that bin, and $\text{conf}(B_m)$ is the mean predicted confidence.

This is also the boundary between `cliare-score-v0` and a future certified score. v0 is appropriate for local CI and improvement tracking. A future v1 score should be frozen only after calibration reports show that the confidence model and score dimensions behave reliably across the benchmark corpus.

12 Relationship to Agent Systems

Agent systems increasingly interleave reasoning and action, and tool use is a central part of that loop [**?**, **?**]. The relevant interface for many software tasks is not only an API schema. It is a command surface with local state, authentication, environment assumptions, filesystem effects, and release drift.

CLIARE artifacts can support agent systems in four ways. First, the shape catalog and command index form a navigation index. They reduce blind exploration and give the agent confidence-weighted command, flag, output, and precondition information. Second, the evidence log is a training corpus. It records how real CLIs expose commands, reject invalid invocations, advertise output formats, require preconditions, and drift across releases. Third, the scorecard is a quality signal. It helps harness builders choose which CLIs are safer to expose to agents and helps maintainers improve the command surfaces agents already use. Fourth, the artifact map is a package manifest for CLIARE output directories. It tells an agent which files exist, which are missing, which schemas they use, whether a run is still active or failed, and where to begin inspection.

The point is not to replace agent planning. It is to give planners a measured interface rather than leaving them to reconstruct the command surface from transient trial and error.

13 Limitations

CLIARE is intentionally conservative in the current implementation. It does not attempt full semantic understanding of every command. It does not run destructive command bodies by default.

It does not claim that all hidden plugin surfaces are discoverable under a fixed budget. It does not treat a high local score as a certified leaderboard result. It does not infer source-level intent from binary behavior.

Large CLIs can exceed quick or standard traversal budgets. Dynamic CLIs can expose different surfaces depending on auth state, installed plugins, current repository, project configuration, and remote service state. Internationalized help, pagers, shell wrappers, and manpage formatting can complicate layout inference. These are not edge cases; they are part of the domain.

The correct response is not to hide uncertainty. CLIARE reports traversal pressure, frontier remaining, stop reason, precondition-blocked paths, confidence, and artifact provenance so a maintainer can decide whether to run a deeper profile, provide fixtures, or improve the CLI surface.

14 Research Agenda

The next technical work falls into five areas: truth sets, calibration, replay and rescore, deeper safety analysis, and harness integration. Truth sets provide human-reviewed labels for commands, flags, arity, output contracts, preconditions, side effects, and known false positives. Calibration tunes evidence weights against those truth sets and evaluates proper scoring rules. Replay and rescore allow score-model changes to be audited without rerunning every probe. Deeper safety analysis expands beyond filesystem side effects toward dry-run detection, destructive verb classification, network behavior, auth scopes, and policy-controlled fixture execution. Harness integration makes shape catalogs directly consumable by agent planners, tool routers, adapter builders, and benchmark systems.

15 Conclusion

CLIs are becoming an operational interface for agents, but the ecosystem lacks a standard way to measure whether a CLI is ready for that role. CLIARE frames CLI-agent-readiness as a runtime measurement problem. It records evidence, infers command shape with probabilistic confidence, scores decomposed readiness dimensions, and integrates with CI so maintainers can improve over time.

The practical value is immediate: detect drift, catch regressions, find parseability gaps, expose unsafe discovery behavior, and give agents a better map. The long-term value is a shared measurement standard for command surfaces that are increasingly central to software automation.

References

[1] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. arXiv:2210.03629, 2022. https://arxiv.org/abs/2210.03629

[2] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language Models Can Teach Themselves to Use Tools. arXiv:2302.04761, 2023. https://arxiv.org/abs/2302.04761

[3] Glenn W. Brier. Verification of Forecasts Expressed in Terms of Probability. Monthly Weather Review, 1950. DOI: 10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2

[4] Tilmann Gneiting and Adrian E. Raftery. Strictly Proper Scoring Rules, Prediction, and Estimation. Journal of the American Statistical Association, 2007. https://doi.org/10.1198/016214506000001437

[5] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On Calibration of Modern Neural Networks. arXiv:1706.04599, 2017. https://arxiv.org/abs/1706.04599

[6] Supply-chain Levels for Software Artifacts. SLSA Provenance, version 1.2. https://slsa.dev/spec/v1.2/provenance

[7] The Open Group. Utility Syntax Guidelines. POSIX Base Specifications, Section 12.2. https://pubs.opengroup.org/onlinepubs/9699919799/basedefs/V1_chap12.html

Target	Score	Probes	Complete	Budget	Machine	Parses	Findings	FS changes
cliare	97.4	23	yes	no	1	1	0	0
rote	63.1	768	no	yes	21	0	4	993
git	92.2	73	yes	no	0	0	1	0
supabase	90.1	403	yes	no	0	0	1	0
gh	87.8	512	no	yes	44	0	1	0
cargo	92.0	152	yes	no	0	0	2	1
npm	36.0	7	yes	no	0	0	2	530
docker	86.3	253	yes	no	0	0	1	0
deno	71.7	512	no	yes	57	0	4	2876

Table 2: Exploratory local real-CLI corpus run. Values are uncalibrated `cliare-score-v0` measurements, not leaderboard results.